NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

STR

LAB #5

freeRTOS: INTERRUPTS & TASK CONTROL

LABWORK 1 ARCHITECTURE TIPS

2022 - 2023

OBJECTIVES

freeRTOS: INTERRUPTS & TASK CONTROL
LABWORK 1 ARCHITECTURE REVIEW

- ❑ Interrupts Review
- ❑ Task Control (task suspend & task resume)
- ❑ Example of Application
- ❑ Labwork 1: Architecture Review & Tips

INTERRUPTS [REVIEW]

- Using Interrupt Service Routines (ISR) on port values

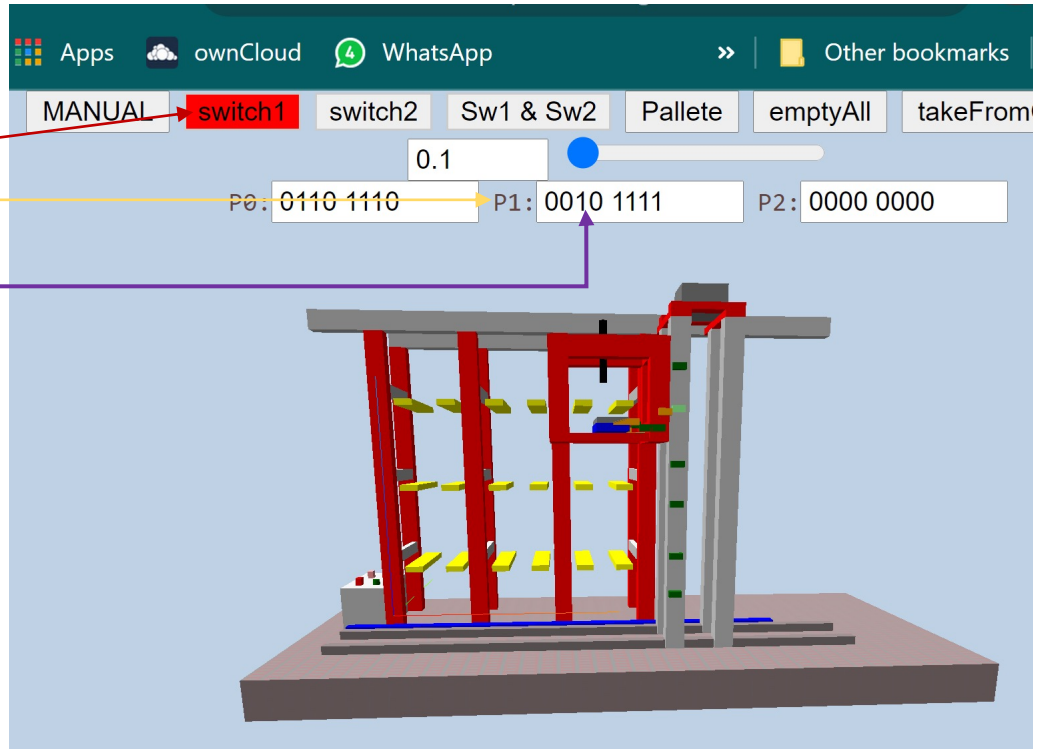
freeRTOS/include -> interrupt.h:

```
void attachInterrupt(int port, int bit, void(*ISR)(ULONGLONG lastTime), int Mode);
```

Example:

switch1 -> p1.5 (active high)

```
void myDaemonTaskStartupHook(void) {
   // …code already here
   attachInterrupt(1, 5, switch1_rising_isr, RISING);
   attachInterrupt(1, 5, switch1_falling_isr, FALLING);
   attachInterrupt(1, 6, switch2_change_isr, CHANGE);
}
```



Mode		
	LOW	Triggers interrupt whenever the pin is LOW
	HIGH	Triggers interrupt whenever the pin is HIGH
	FALLING	Triggers interrupt when the pin goes from HIGH to LOW
	RISING	Triggers interrupt when the pin goes from LOW to HIGH
	CHANGE	Triggers interrupt whenever the pin changes value, from HIGH to LOW or LOW to HIGH

INTERRUPTS [REVIEW]

- Using Interrupt Service Routines (ISR) on port values

```
void attachInterrupt(int port, int bit, void(*ISR)(ULONGLONG lastTime), int Mode);
```

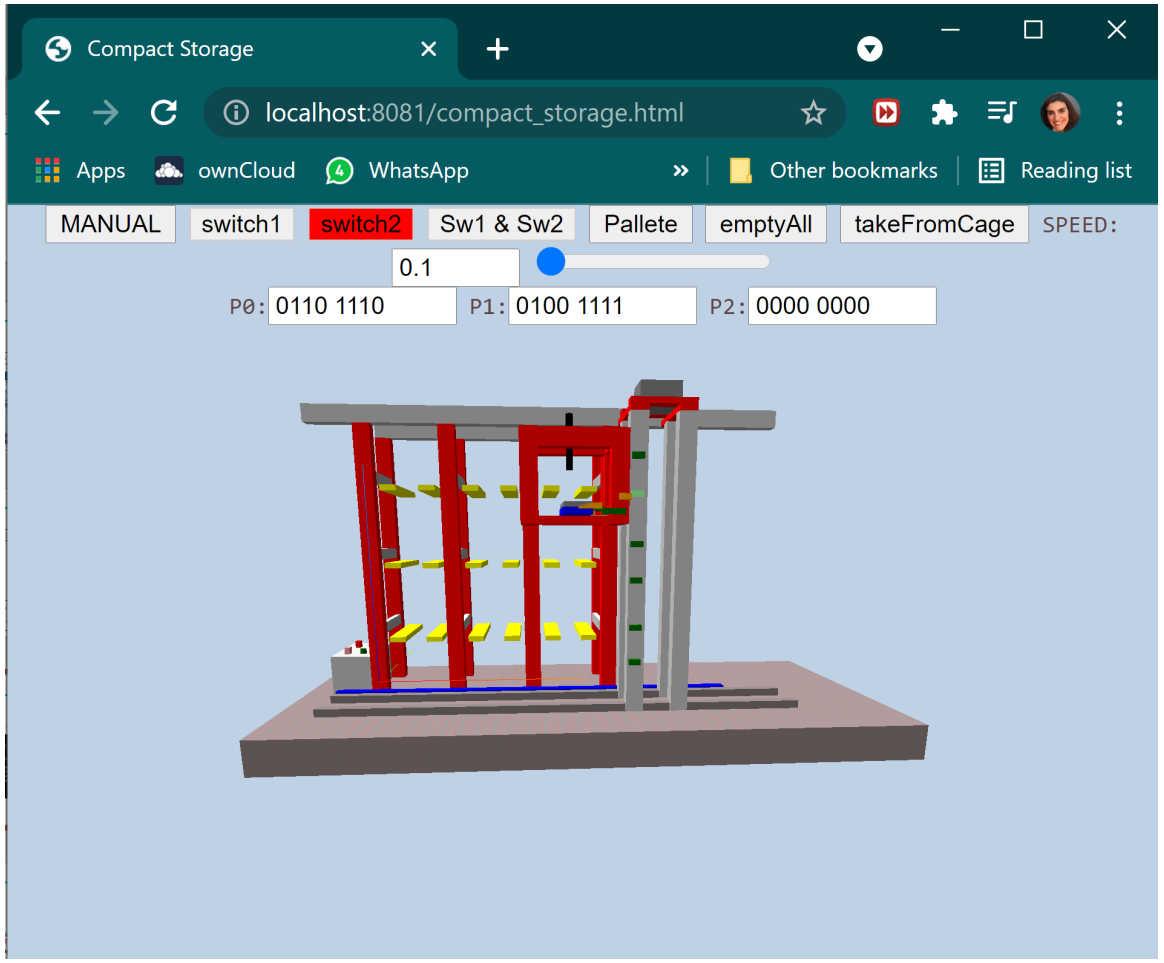
```
void myDaemonTaskStartupHook(void) {
   // …code already here
   attachInterrupt(1, 5, switch1_rising_isr, RISING);
   attachInterrupt(1, 5, switch1_falling_isr, FALLING);
   attachInterrupt(1, 6, switch2_change_isr, CHANGE);
}
```

Interrupt routine

```
void switch1_rising_isr(ULONGLONG lastTime) {
    // GetTickCount64() current time in miliseconds
    // since the system has started...
    ULONGLONG  time = GetTickCount64();
    printf("\nSwitch one RISING detected at time = %llu...\n", time);
}
```

INTERRUPTS [REVIEW]

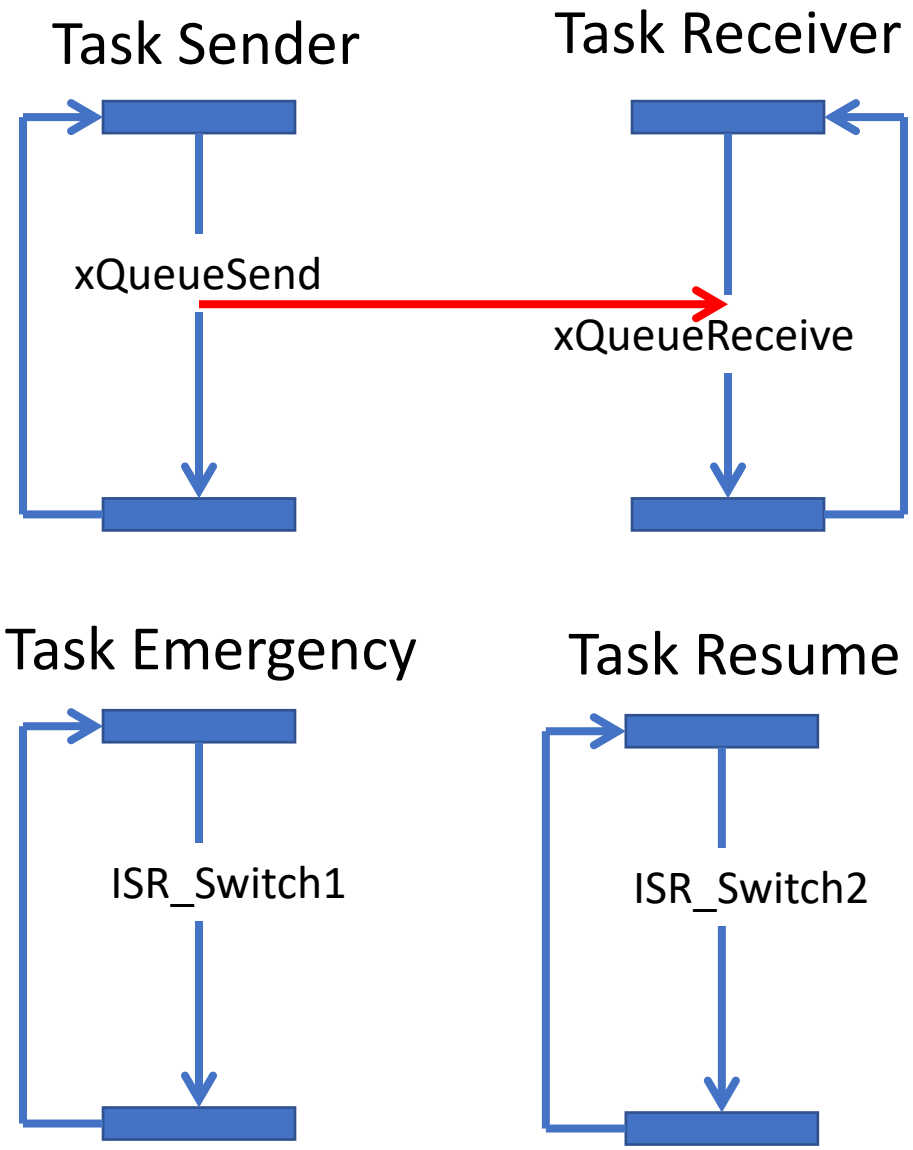
- Using Interrupt Service Routines (ISR) on port values



Running example on demo_rtos project
(from **2021_22_STR_TLAB1_3.pptx**)

```
c:\str\demo_rtos\Debug\demo_rtos.exe

waiting for hardware simulator...
Reminding: gotoXZ requires kit calibration first...
Starting web server on port 8081

got access to simulator...write something

Switch one RISING detected at time = 109690140...
Switch one FALLING detected at time = 109690531...
Switch two CHANGE detected at time = 109692890...
Switch two CHANGE detected at time = 109692906...
vTask_waits is tired of waiting
```

INTERRUPTS & TASK CONTROL



- Two parallel tasks exchanging data through a MAILBOX
- One emergency task triggered by an interrupt -> switch 1 (suspends task sender)
- One resume task triggered by an interrupt -> switch 2 (resumes task sender)

INTERRUPTS & TASK CONTROL

- Let's update project **demo_rtos.sln** (in C:\str\demo_rtos) to try this example:

```
// tasks handler declaration
xTaskHandle emergencyTask;
xTaskHandle resumeTask;
xTaskHandle taskSender;
```

Emergency and Resume Tasks Creation & Implementation

```
void vTaskEmergency(void* pvParameters){
    // The task being suspended and resumed.
    for (;; ){
       // The task suspends itself.
       vTaskSuspend(NULL);
       // The task is now suspended, so will
       not reach here until the ISR resumes it.
       printf("\n **** EMERGENCY task\n");
       // The task suspends task sender.
       vTaskSuspend(taskSender);
    }
}
```

```
void vTaskResume(void* pvParameters){
    // The task being suspended and resumed.
    for (;; ){
       // The task suspends itself.
       vTaskSuspend(NULL);
       // The task is now suspended, so will
       not reach here until the ISR resumes it.
       printf("\n **** RESUME task\n");
       // The task suspends task sender.
       vTaskResume(taskSender);
    }
```

The handler of the **taskSender** needs to be known!

```
void myDaemonTaskStartupHook(void) {
    inicializarPortos();
    // *** some code here
    xTaskCreate(vTaskEmergency, "vTaskEmergency ", 100, NULL, 0, &emergencyTask);
    xTaskCreate(vTaskResume, "vTaskResume ", 100, NULL, 0, &resumeTask);
}
```

The handler of the task will be needed by the ISR!

INTERRUPTS & TASK CONTROL

ISR Routines Implementation

```
void switch1_rising_isr(ULONGLONG lastTime) {
    // GetTickCount64() current time in miliseconds
    // since the system has started...
    ULONGLONG  time = GetTickCount64();
    printf("\nSwitch one RISING detected at time = %llu...\n", time);

    BaseType_t xYieldRequired;
    // Resume the suspended task.
    xYieldRequired = xTaskResumeFromISR(emergencyTask);
}
```

```
void switch2_rising_isr(ULONGLONG lastTime) {
    ULONGLONG  time = GetTickCount64();
    printf("\nSwitch two RISING detected at time = %llu...", time);

    BaseType_t xYieldRequired;
    // Resume the suspended task.
    xYieldRequired = xTaskResumeFromISR(resumeTask);
}
```

INTERRUPTS & TASK CONTROL

In myDaemonTaskStartupHook…

```
void myDaemonTaskStartupHook(void) {
    inicializarPortos();

    // *** some code here

    xQueueHandle xQueue = xQueueCreate(10, sizeof(double));
    xTaskCreate(vTaskCode_MbxReceiver, "vTaskCode_MbxReceiver ", 100, xQueue, 0, &taskReceiver);
    xTaskCreate(vTaskCode_MbxSender, "vTaskCode_MbxSender ", 100, xQueue, 0, &taskSender);

    xTaskCreate(vTaskEmergency, "vTaskEmergency ", 100, NULL, 0, &emergencyTask);
    xTaskCreate(vTaskResume, "vTaskResume ", 100, NULL, 0, &resumeTask);

    attachInterrupt(1, 5, switch1_rising_isr, RISING);
    attachInterrupt(1, 6, switch2_rising_isr, RISING);

}
```

INTERRUPTS & TASK CONTROL

C:\str\demo_rtos\Debug\demo_rtos.exe

```
waiting for hardware simulator...
Reminding: gotoXZ requires kit calibration first.
Starting web server on port 8081

got access to simulator...
received_val =1.000
received_val =2.000
received_val =3.000
received_val =4.000
received_val =5.000
received_val =6.000
received_val =7.000
received_val =8.000
received_val =9.000
Switch one RISING detected at time = 9057
 **** EMERGENCY task
```

Compact Storage

localhost:8081/compact_st...

Apps ownCloud » Other bookmarks Reading list

MANUAL switch1 switch2 Sw1 & Sw2 Pallete emptyAll

takeFromCage SPEED: 0.1

P0: 1110 1111 P1: 0010 1111 P2: 0000 0000

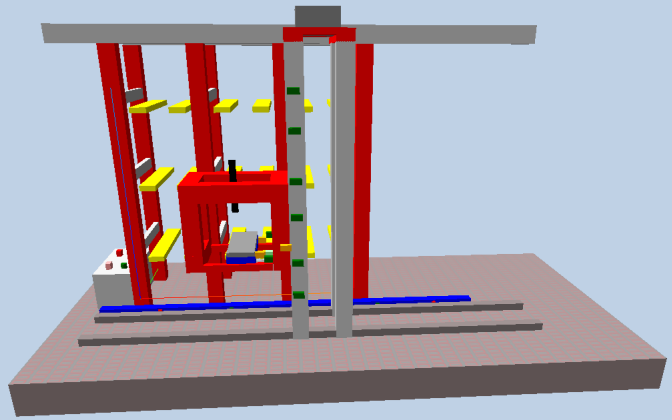
```
eceived_val =1.000
eceived_val =2.000
eceived_val =3.000
eceived_val =4.000
eceived_val =5.000
eceived_val =6.000
eceived_val =7.000
eceived_val =8.000
eceived_val =9.000
witch one RISING detected at time = 90579531...

 **** EMERGENCY task

witch two CHANGE detected at time = 90694171...
 **** RESUME task

eceived_val =10.000
eceived_val =11.000
witch two CHANGE detected at time = 90699062...
 **** RESUME task

eceived_val =12.000
```

Apps ownCloud » Other bookmarks Reading list

MANUAL switch1 switch2 Sw1 & Sw2 Pallete emptyAll

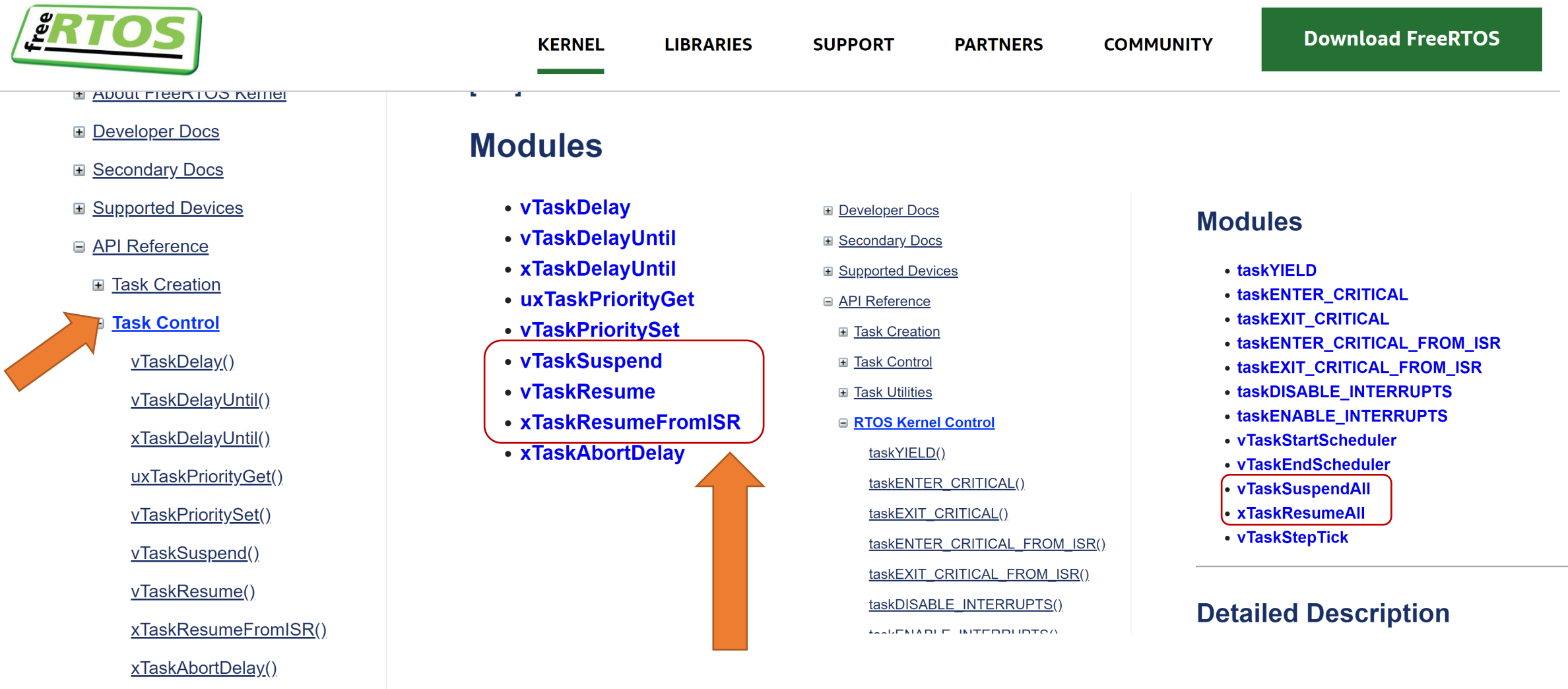
takeFromCage SPEED: 0.1

P0: 1110 1111 P1: 0100 1111 P2: 0000 0000



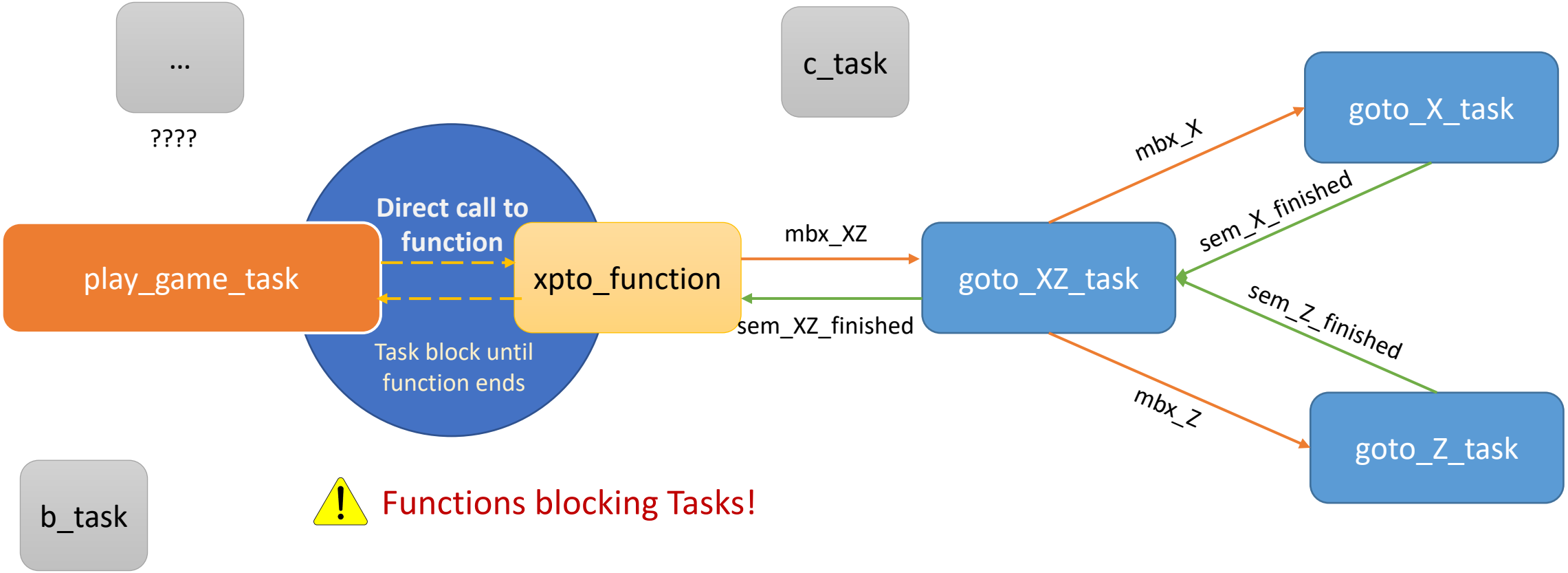
Running the project...

freeRTOS API – TASK CONTROL



Lab Work 1 – Architecture Tips

1. Calling a function from task play_game…

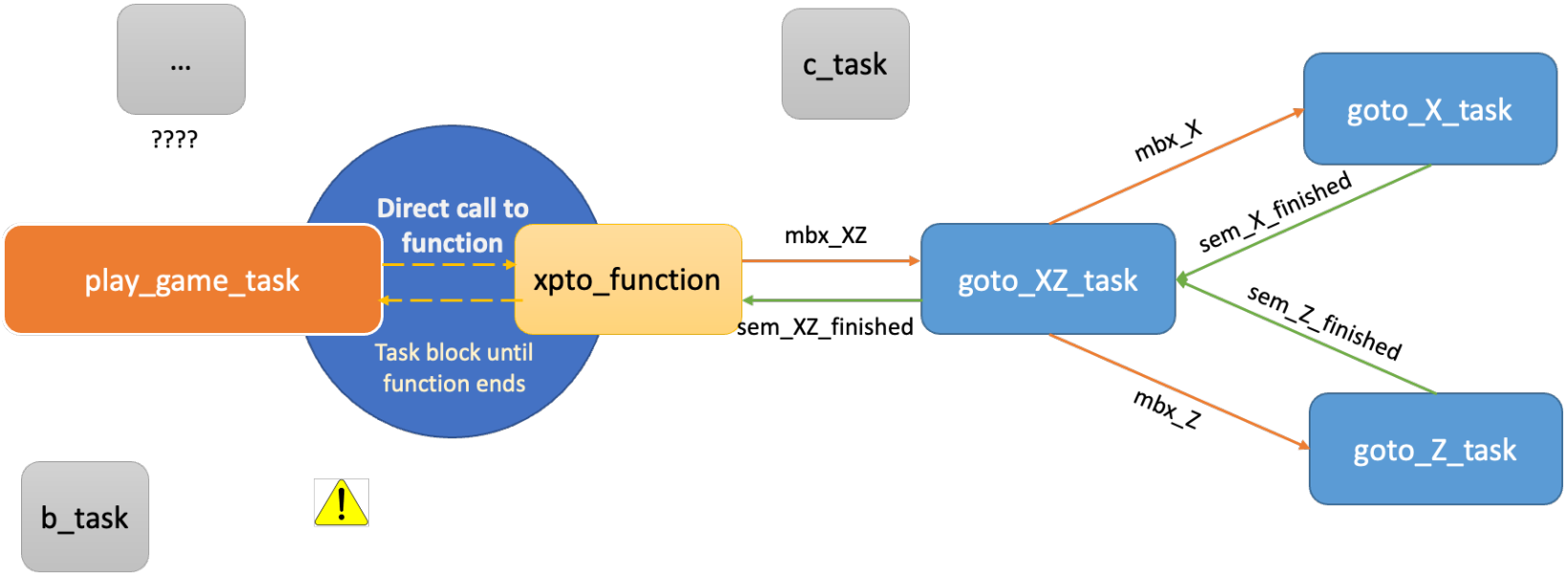


If there are more tasks being performed at the same time…. Called from play_game_task…..
They will have to wait that xpto function finishes!

Lab Work 1 – Architecture Tips

Calling a function from task play_game…

- ✓ Calling a function has the effect of blocking **play_game_task** until the function finishes its work. During this state, **play_game_task** is not able to receive new requests and therefore can't work concurrently, react and "invoke" other tasks.

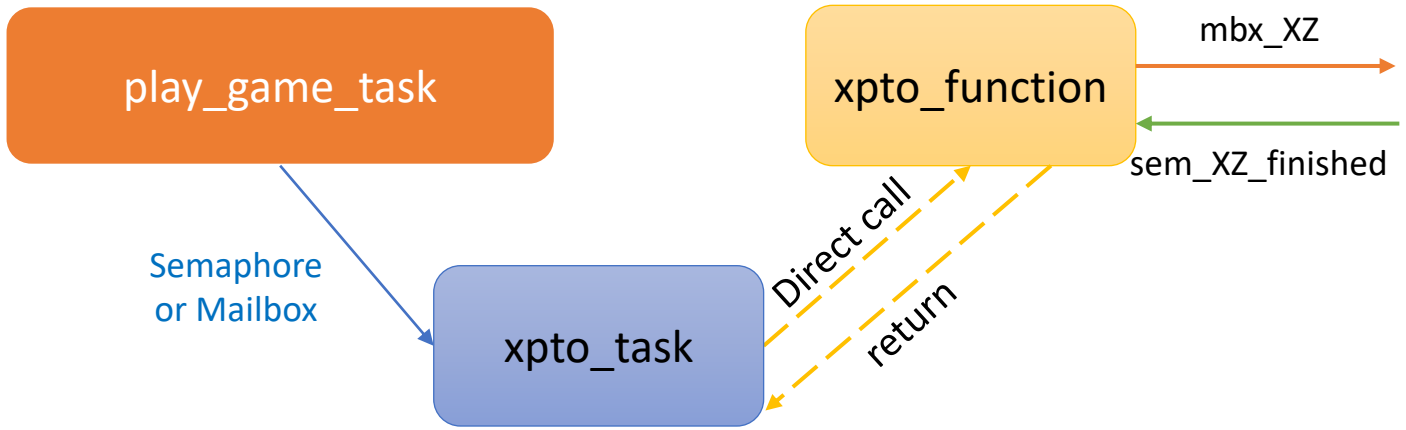


How to solve it?

Lab Work 1 – Architecture Tips

❑ Invoke the xpto_function from a **new task**!

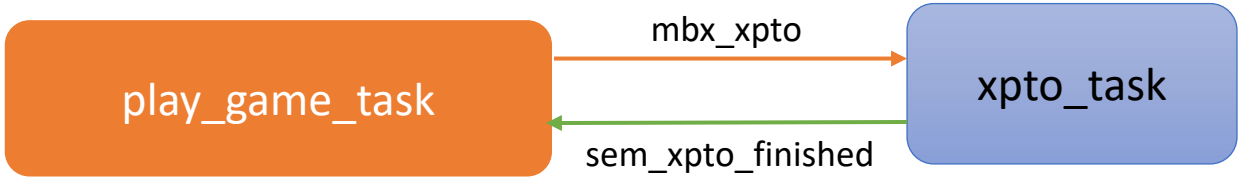
- Create an intermediate task that takes the role of waiting for a request to call **xpto_function**.
- Use synchronization or messages, as appropriate, to notify **xpto_task**.
- This way, as opposite to the direct call of **xpto_function**, play_game_task can invoke it indirectly, without becoming blocked.



Lab Work 1 – Architecture Tips

2. Task blocking another task…

- In this case, the **play_game_task** sends a message requesting a service and then waits for sem_xpto_finished.
- The problem illustrated in the picture, is that **play_game_task** is still blocked by semaphore sem_xpto_finished until the **task_XPTO** is finished, and so, preventing/impeding the request of other services.

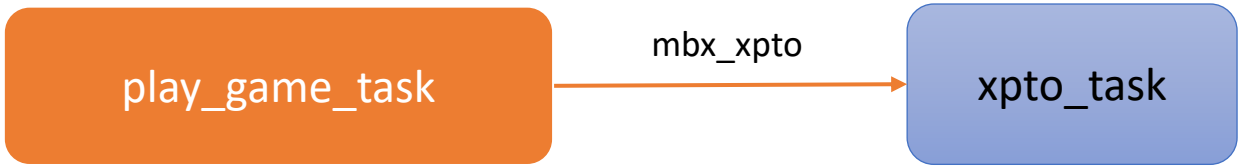


How to solve it?

Lab Work 1 – Architecture Tips

a) Consider removing the semaphore! 

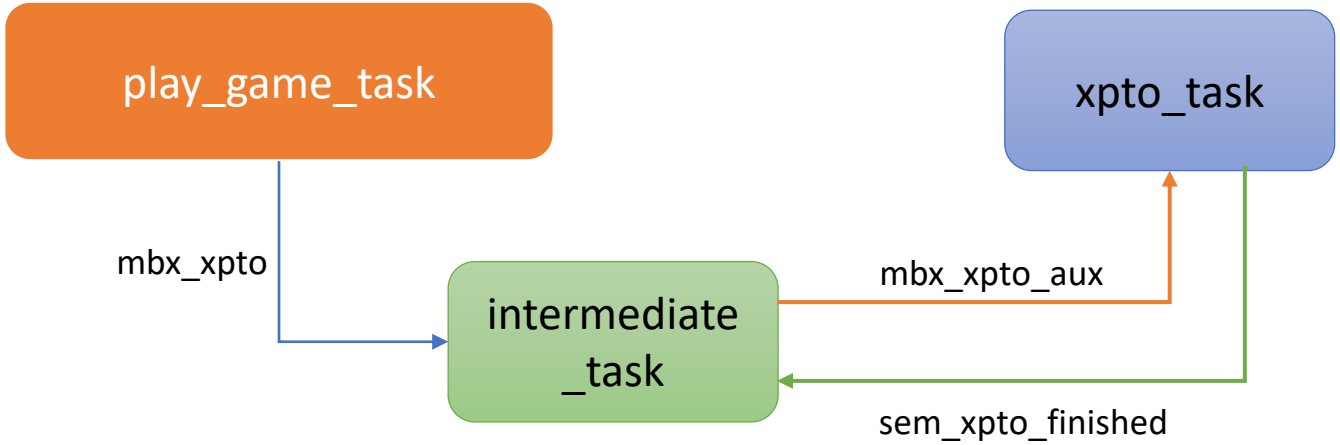
- First check whether you really need the semaphore sem_xpto_finished.
- If it is not necessary, then the task **play_game_task** just sends the message instantaneously and proceeds to be available for other requests.



Lab Work 1 – Architecture Tips

b) Use an **intermediate task**!

- If a semaphore is indeed needed, the solution is to use a mediation (intermediate) task, which is assigned with the role of effectively invoking the service implementation and then waiting for its termination.
- In this way, **play_game_task** does not block waiting for the semaphore, therefore, being available to receive more requests or/and user interaction.



KEEP UP THE GOOD WORK!